

IEEE 830

Especificación de Requisitos de Software (ERS)

Sistema Bancario con Detección de Fraude y Scoring Crediticio

Versión	2.0
Fecha	17 de junio de 2026
Equipo	Taritolay, Nicolás Daniel - Aramayo, Enzo Demian
Estándar	IEEE Std 830-1998

Tabla de Contenidos

- 1. Introducción**
 - 1.1 Propósito
 - 1.2 Alcance del Sistema
 - 1.3 Definiciones, Acrónimos y Abreviaturas
 - 1.4 Referencias
 - 1.5 Visión General del Documento
- 2. Descripción General**
 - 2.1 Perspectiva del Producto
 - 2.2 Funciones del Producto
 - 2.3 Características de los Usuarios
 - 2.4 Restricciones
 - 2.5 Suposiciones y Dependencias
- 3. Requisitos Específicos**
 - 3.1 Requisitos Funcionales
 - 3.2 Requisitos No Funcionales
 - 3.3 Arquitectura del Sistema
- 4. Apéndices**
 - 4.1 Modelo de Datos (DER Resumido)
 - 4.2 Stack Tecnológico
 - 4.3 Estado Actual y Próximos Pasos

1. Introducción

1.1 Propósito

Este documento especifica los requisitos funcionales y no funcionales del Sistema Bancario con Detección de Fraude y Scoring Crediticio, un software web desarrollado como proyecto académico. El sistema simula un entorno bancario integral que permite a los usuarios registrarse, iniciar sesión, gestionar cuentas con alias y CVU, realizar transferencias electrónicas, solicitar préstamos con análisis crediticio automatizado, constituir plazos fijos, e incorporar mecanismos de concurrencia y un módulo de minería de datos para la detección de fraudes y evaluación de riesgo crediticio.

El presente documento sigue la estructura recomendada por el estándar IEEE 830-1998.

1.2 Alcance del Sistema

El sistema abarca los siguientes módulos funcionales:

Módulo	Estado
Módulo 1: Autenticación y gestión de clientes	Implementado
Módulo 2: Cuentas bancarias con alias y CVU	Implementado
Módulo 3: Transferencias con concurrencia	Implementado
Módulo 4: Detección de fraude con ML	En Desarrollo
Módulo 5: Seguridad OWASP	Implementado
Módulo 6: Pruebas de concurrencia	En Desarrollo
Módulo 7: Préstamos con scoring crediticio (ML)	Implementado
Módulo 8: Plazo fijo	En Desarrollo
Módulo 9: Tarjetas de débito/crédito (solo modelo)	En Desarrollo
Módulo 10: Panel de administración personalizado	En Desarrollo

1.3 Definiciones, Acrónimos y Abreviaturas

Término	Definición
ERS	Especificación de Requisitos de Software (IEEE 830)
MTV	Modelo-Template-View (patrón arquitectónico de Django)
Hexagonal	Arquitectura de Puertos y Adaptadores
ML	Machine Learning (aprendizaje automático)
ACID	Atomicity, Consistency, Isolation, Durability
OWASP	Open Web Application Security Project
WCAG	Web Content Accessibility Guidelines
CSRF	Cross-Site Request Forgery
XSS	Cross-Site Scripting
2FA	Autenticación de dos factores

Locust	Herramienta de pruebas de carga
ORM	Object-Relational Mapping
CVU	Clave Virtual Uniforme (identificador de cuenta)
Alias	Nombre simplificado para identificar una cuenta
PCI DSS	Payment Card Industry Data Security Standard
OTP	One-Time Password
DER	Diagrama de Entidad-Relación

1.4 Referencias

1. IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.
2. WCAG 2.1: Web Content Accessibility Guidelines. W3C.
3. OWASP Top Ten: Proyecto de seguridad en aplicaciones web.
4. Documentación oficial de Django 6.0: <https://docs.djangoproject.com/>
5. PCI DSS v4.0: Payment Card Industry Data Security Standard.

1.5 Visión General del Documento

El documento se estructura en tres secciones principales:

6. Introducción: Propósito, alcance y definiciones.
7. Descripción General: Perspectiva del producto, funciones y restricciones.
8. Requisitos Específicos: Requisitos funcionales detallados, no funcionales y arquitectura.

2. Descripción General

2.1 Perspectiva del Producto

El sistema es una aplicación web desarrollada con Django 6.0 (framework Python) y base de datos PostgreSQL. Sigue una arquitectura híbrida que combina:

- MTV (Modelo-Template-View) de Django para el módulo de autenticación y la interfaz de usuario.
- Arquitectura Hexagonal (puertos y adaptadores) para el núcleo transaccional y el motor de minería de datos.

2.2 Funciones del Producto

Función	Descripción
Registro de clientes	Captura datos personales (DNI, fecha de nacimiento, género, profesión, ingreso mensual, nivel educativo) y crea una cuenta de usuario.
Inicio de sesión	Autenticación segura con contraseña hasheada.
Dashboard	Consulta de saldo, movimientos recientes y resumen de productos bancarios.
Transferencias	Envío de dinero entre cuentas con bloqueo concurrente y detección de fraude en tiempo real.
Alias y CVU	Asignación de alias personalizado y CVU único a cada cuenta.
Tarjetas de débito/crédito	Modelo de tarjetas (gestión pendiente), sin almacenamiento de CVV (PCI DSS).
Préstamos personales	Solicitud, evaluación crediticia automática mediante ML, desembolso y plan de pagos.
Plazo fijo	Constitución, consulta de intereses y cancelación anticipada.
Detección de fraude	Evaluación de cada transacción mediante motor de ML con generación de alertas.
Seguridad y accesibilidad	Cumplimiento de OWASP Top 10 y WCAG 2.1 AA.

2.3 Características de los Usuarios

Tipo de Usuario	Descripción
Cliente	Usuario final que se registra, consulta su saldo, realiza transferencias, solicita préstamos y gestiona plazos fijos.
Administrador	Accede al panel de Django Admin para gestionar usuarios, cuentas, transacciones y alertas.

2.4 Restricciones

- Tecnológicas: Django 6.0, PostgreSQL, Python 3.10+.
- Arquitectónicas: Híbrida MTV + Hexagonal.
- Seguridad: Cumplir con OWASP Top 10, WCAG 2.1 AA y PCI DSS (sin almacenamiento de CVV).
- Académicas: Entregar documento IEEE 830, pruebas de concurrencia y módulo de minería de datos.

2.5 Suposiciones y Dependencias

- Los usuarios disponen de un navegador web moderno.
- La base de datos PostgreSQL está disponible (local o en Neon).
- El modelo de ML se entrena con datos sintéticos generados por el equipo.

3. Requisitos Específicos

3.1 Requisitos Funcionales

RF-01: Registro de Cliente

ID	RF-01
Prioridad	Alta
Estado	Implementado

Descripción

El sistema permite a un usuario no autenticado crear una cuenta proporcionando: nombre de usuario, nombre, apellido, correo electrónico, DNI, fecha de nacimiento, género, profesión, ingreso mensual, nivel educativo, teléfono, dirección y contraseña.

Validación

- Contraseñas coincidentes.
- Formato de email válido.
- Unicidad de username y DNI.
- DNI: 7 u 8 dígitos numéricos (validado con RegexValidator tanto en modelo como en formulario).
- La validación se ejecuta tanto a nivel de modelo como de formulario (Django no ejecuta los validators del modelo automáticamente en save(), por lo que se duplican en el formulario).

Resultado

Se crea un User de Django y un perfil Cliente asociado mediante una señal post_save.

RF-02: Inicio de Sesión

ID	RF-02
Prioridad	Alta
Estado	Implementado

Descripción

Un usuario registrado puede iniciar sesión con su nombre de usuario y contraseña.

Seguridad

- Sesiones firmadas.
- Protección CSRF en todos los formularios.
- Contraseña hasheada con PBKDF2 (default de Django).

Resultado

Redirección al dashboard personal del cliente.

RF-03: Página de Inicio Pública

ID	RF-03
----	-------

Prioridad	Media
Estado	Implementado

Descripción

Landing page accesible desde la raíz del sitio (/) que muestra información institucional del banco, hero con mensaje de bienvenida, tres tarjetas de características principales, y botones para iniciar sesión o registrarse.

Accesibilidad

- Contraste de color WCAG 2.1 AA.
- Navegación por teclado.
- HTML semántico con landmarks ARIA.

RF-04: Consulta de Saldo (Dashboard)

ID	RF-04
Prioridad	Alta
Estado	Implementado

Descripción

Un usuario autenticado puede ver el saldo actual de su cuenta, el alias y CVU asociados, un resumen de productos (tarjetas, préstamos activos, plazos fijos) y los movimientos recientes en una página de dashboard personalizada.

Datos mostrados

- Saldo disponible.
- Alias y CVU de la cuenta.
- Últimos 10 movimientos con fecha, descripción, monto y saldo resultante.
- Resumen de tarjetas asociadas.
- Estado de préstamos activos.

RF-05: Transferencia Bancaria

ID	RF-05
Prioridad	Alta
Estado	Implementado

Descripción

Un usuario autenticado puede transferir dinero de su cuenta a otra cuenta del mismo banco. La operación requiere seleccionar la cuenta destino (por alias, CVU o número de cuenta), ingresar el monto y una descripción (opcional).

Validación

- Saldo suficiente en la cuenta origen.
- Cuenta destino existente.
- Monto positivo.
- Límite diario configurable.

Concurrencia

La operación se ejecuta dentro de una transacción ACID con SELECT FOR UPDATE (bloqueo pesimista), garantizando que dos transferencias simultáneas no causen sobregiro.

Detección de fraude

Se prevé la integración futura con el motor de ML para evaluar cada transferencia

Resultado

Se descuenta el monto de la cuenta origen, se acredita en la cuenta destino, y se registra la transacción en el historial de ambas cuentas.

RF-06: Historial de Movimientos

ID	RF-06
Prioridad	Media
Estado	Implementado

Descripción

El usuario puede consultar el historial completo de transacciones de su cuenta, con paginación (10 movimientos por página), filtros por fecha y tipo de operación, y opción de exportar a CSV.

RF-07: Detección de Fraude (En Desarrollo)

ID	RF-07
Prioridad	Alta
Estado	En Desarrollo

Descripción

Cada transferencia es evaluada por un motor de ML que asigna un score de riesgo (0.0 a 1.0). Si supera un umbral configurable, se genera una AlertaFraude y la operación queda pendiente de revisión.

Modelo

Random Forest entrenado con características de la transacción (monto, horario, frecuencia, ubicación geográfica, historial del cliente) mediante Scikit-learn.

Almacenamiento

Las alertas se persisten en el modelo AlertaFraude con estado pendiente, revisada o resuelta.

Integración

A través del puerto MotorFraude implementado por un adaptador que carga el modelo serializado (SklearnFraudAdapter).

RF-08: Autenticación de Dos Factores (2FA) (En Desarrollo)

ID	RF-08
Prioridad	Media
Estado	En Desarrollo

Descripción

Para transferencias de alto monto (superiores a un umbral configurable), el usuario debe ingresar un código OTP enviado a su correo electrónico o generado por una app de autenticación.

Configuración

El cliente puede activar o desactivar el 2FA desde su perfil. El estado se persiste en ConfiguracionSeguridad.doble_factor_activo.

RF-09: Gestión de Alias y CVU (Implementado)

ID	RF-09
Prioridad	Alta
Estado	Implementado

Descripción

Cada cuenta puede tener un alias (nombre corto definido por el usuario, único en el sistema) y un CVU (Clave Virtual Uniforme de 22 dígitos generada automáticamente) que permiten identificar la cuenta en transferencias sin revelar el número de cuenta real.

Restricciones

- El alias debe ser único, de entre 6 y 20 caracteres alfanuméricos.
- El CVU se calcula en base al estándar argentino (BCRA).

Operaciones

- Alta de alias y CVU al crear la cuenta.
- Modificación del alias.
- Consulta de alias/CVU en el dashboard.

RF-10: Solicitud de Préstamo (Implementado)

ID	RF-10
Prioridad	Alta
Estado	Implementado

Descripción

Un cliente autenticado puede solicitar un préstamo personal indicando el monto deseado, el plazo en meses y el sistema de amortización (francés o alemán). El sistema ejecuta un análisis crediticio automático (ML) que evalúa el riesgo del cliente basado en su historial de transacciones, saldos promedio, ingresos estimados y comportamiento previo.

Reglas

- El monto solicitado no puede exceder el límite de crédito calculado por el scoring.
- Si la evaluación es positiva, se aprueba el préstamo y se acredita el monto en la cuenta.
- El préstamo se registra en el modelo Prestamo con su plan de cuotas asociado.

RF-11: Simulación y Plan de Pagos del Préstamo (Implementado)

ID	RF-11
Prioridad	Media
Estado	Implementado

Descripción

Antes de confirmar la solicitud, el sistema muestra una simulación con el detalle de cuotas mensuales, tasa de interés aplicada, monto total a pagar, y fecha de vencimiento de cada cuota.

Cálculo

- Sistema de amortización francés (cuota fija).
- Sistema de amortización alemán (cuota decreciente).
- El usuario puede elegir entre ambos sistemas y ver la comparativa antes de confirmar.

RF-12: Gestión de Cuotas de Préstamo (Implementado)

ID	RF-12
Prioridad	Media
Estado	Implementado

Descripción

El cliente puede consultar el estado de sus cuotas (pendientes, pagadas, vencidas) y realizar el pago manual de una cuota desde su cuenta. El sistema registra cada pago y actualiza el saldo del préstamo.

Funcionalidades

- Calendario de pagos con fechas de vencimiento.
- Pago anticipado de cuotas.
- Débito automático configurable.
- Notificaciones de cuotas próximas a vencer.

RF-13: Constitución de Plazo Fijo (En Desarrollo)

ID	RF-13
Prioridad	Media
Estado	En Desarrollo

Descripción

El cliente puede crear un plazo fijo seleccionando la cuenta de débito, el monto a inmovilizar y el plazo en días. El sistema calcula los intereses según una tasa fija predefinida y muestra el monto al vencimiento.

Validación

- Saldo suficiente en la cuenta origen.
- Plazo mínimo y máximo configurables.

Efecto

- El monto se debita de la cuenta y se acredita a una cuenta virtual de plazo fijo.
- Al vencimiento, se acredita el capital más intereses a la cuenta origen.

RF-14: Cancelación Anticipada de Plazo Fijo (En Desarrollo)

ID	RF-14
Prioridad	Baja
Estado	En Desarrollo

Descripción

El cliente puede cancelar un plazo fijo antes del vencimiento, recibiendo el capital original sin intereses (o con una penalización predefinida según las políticas del banco).

RF-15: Scoring Crediticio — Módulo ML (En Desarrollo)

ID	RF-15
Prioridad	Alta
Estado	En Desarrollo

Descripción

El motor de scoring crediticio es un modelo de ML (Random Forest o Gradient Boosting) que toma como entrada variables del cliente y su comportamiento histórico para predecir la probabilidad de impago.

Variables de entrada

- Edad de la cuenta.
- Saldo promedio.
- Regularidad de ingresos.
- Cantidad de transacciones.
- Uso de límites.
- Historial de préstamos previos.

Salida

- Puntaje de crédito (0 a 1, donde 1 es máximo riesgo).
- Clasificación categórica (A, B, C, D).

Integración

A través del puerto MotorScoring implementado por un adaptador que carga el modelo serializado.

Almacenamiento

El score crediticio inicial del cliente se persiste en Cliente.score_crediticio_inicial.

3.2 Requisitos No Funcionales

RNF-01: Concurrencia (En Desarrollo)

ID	RNF-01
Estado	En Desarrollo

Descripción

El sistema soporta al menos 50 peticiones simultáneas de transferencia sobre la misma cuenta sin alterar la integridad del saldo.

Verificación

Prueba de carga con Locust simulando 50 usuarios concurrentes.

Implementación

Uso de `select_for_update()` dentro de transacciones atómicas de Django para bloqueo pesimista a nivel de base de datos.

RNF-02: Seguridad (OWASP)

ID	RNF-02
Estado	Implementado

Descripción

Se implementaron protecciones contra las principales vulnerabilidades OWASP:

Vulnerabilidad	Protección
CSRF	Token CSRF en todos los formularios ({% csrf_token %})
XSS	Escape automático de variables en templates Django
SQL Injection	ORM de Django (todas las consultas parametrizadas)
Fuerza Bruta	Rate limiting en formularios de login
Exposición de secretos	Credenciales en .env (excluido del repositorio)
Almacenamiento de CVV	Eliminado del modelo Tarjeta (PCI DSS)
Contraseñas	Hasheadas con PBKDF2 (default Django)

RNF-03: Accesibilidad (WCAG 2.1 AA)

ID	RNF-03
Estado	Implementado

Descripción

La interfaz cumple con las pautas WCAG 2.1 Nivel AA:

- Contraste de color: Paleta azul marino sobre fondo claro, verificado con herramientas de contraste.

- Etiquetas ARIA: Navegación con aria-label en header, footer y secciones principales.
- Navegación por teclado: Todos los formularios y enlaces son accesibles mediante teclado (Tab, Enter).
- Manejo de errores: Mensajes de error visibles y descriptivos en formularios, con aria-describedby.
- HTML semántico: Uso de <header>, <main>, <nav>, <footer>, <section>, <article>.

RNF-04: Disponibilidad y Rendimiento (En Desarrollo)

ID	RNF-04
Estado	En Desarrollo

Descripción

El sistema responde a solicitudes HTTP en menos de 2 segundos bajo carga normal. La base de datos PostgreSQL en Neon escala a cero cuando no está en uso.

Estrategias

- Conexión persistente a la base de datos.
- Optimización de consultas ORM con select_related() y prefetch_related().
- Cache de templates para páginas públicas.

RNF-05: Mantenibilidad (En Desarrollo)

ID	RNF-05
Estado	En Desarrollo

Descripción

El código sigue los principios SOLID y la arquitectura hexagonal permite cambiar componentes externos (base de datos, motor ML, sistema de autenticación) sin modificar la lógica de negocio.

Patrones aplicados

Patrón	Aplicación
Repository	RepositorioCuenta → DjangoCuentaRepository (acceso a BD con select_for_update)
Strategy	MotorFraude → SklearnFraudAdapter (intercambiar motor de fraude)
Unit of Work	@transaction.atomic (transacciones ACID)
Factory	Creación de Transaccion con estado inicial válido
GRASP Controlador	RealizarTransferencia como caso de uso central

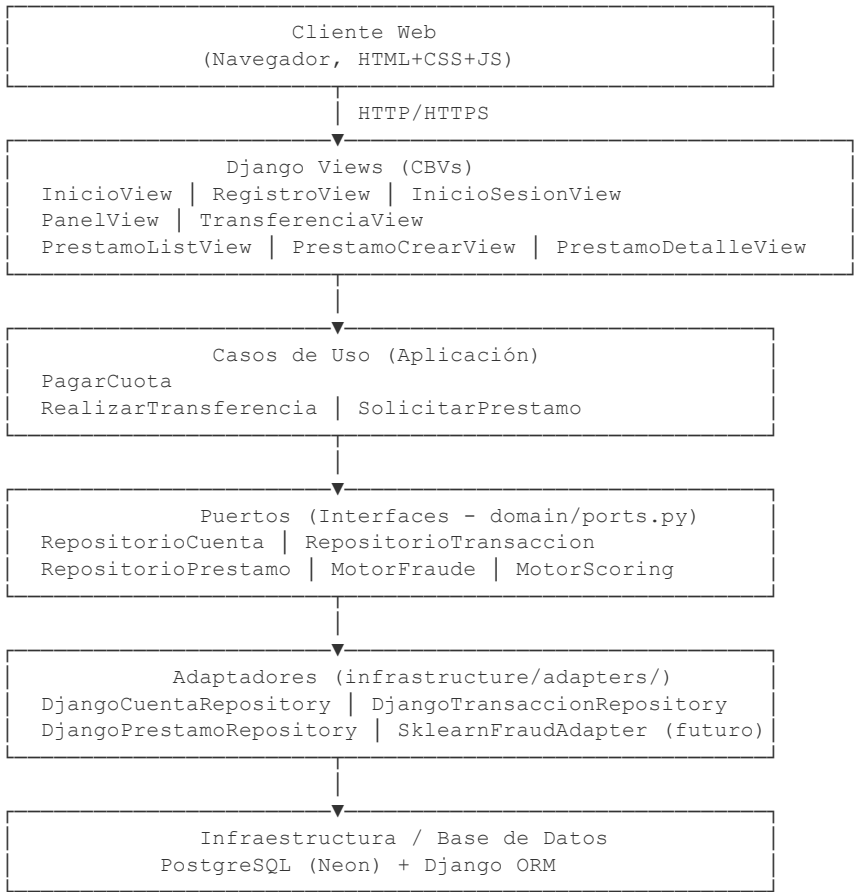
3.3 Arquitectura del Sistema

3.3.1 Estilo Arquitectónico

Arquitectura híbrida MTV + Hexagonal:

Capa	Estilo	Descripción
Autenticación	MTV (Django clásico)	Vistas, formularios y plantillas estándar para registro, login y dashboard.
Core bancario	Hexagonal (implementado)	Entidades puras en domain/entities.py, puertos en domain/ports.py, reglas en domain/rules.py, casos de uso en application/use_cases.py, adaptadores Django en infrastructure/adapters/repositories.py.
Motor ML	Hexagonal	Puertos para detección de fraude y scoring crediticio, implementados por adaptadores que cargan modelos serializados.

3.3.2 Diagrama de Componentes



3.3.3 Patrones de Diseño Aplicados

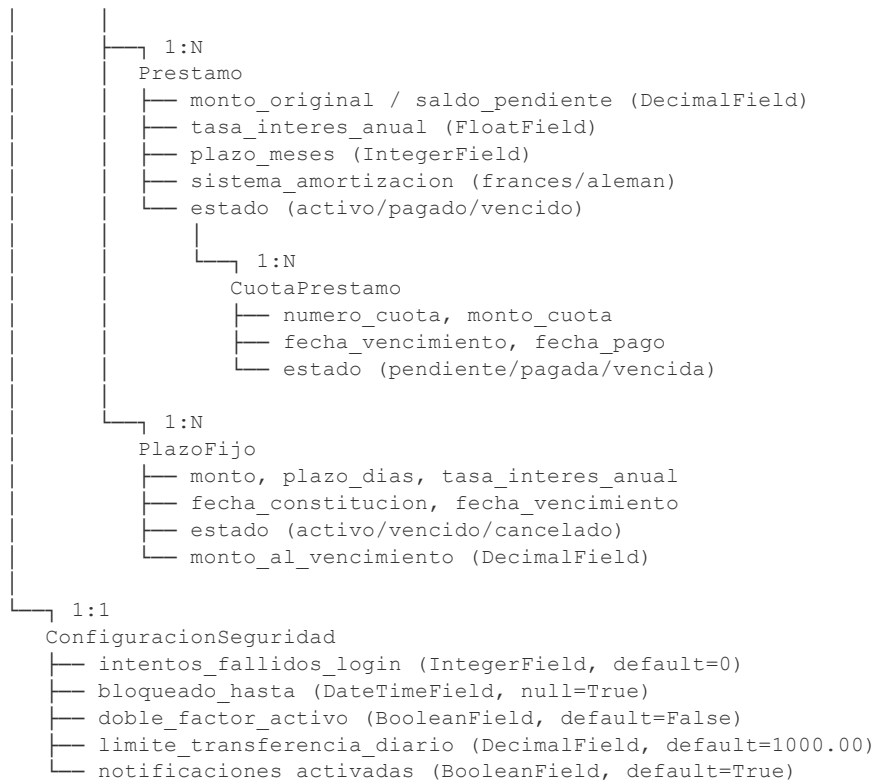
Patrón	Aplicación
--------	------------

Repository	RepositorioCuenta define la interfaz; DjangoCuentaRepository implementa el acceso a BD con select_for_update() para concurrencia.
Strategy	MotorFraude es la interfaz; SklearnFraudAdapter y ReglaFraudeAdapter son implementaciones intercambiables.
Unit of Work	Django @transaction.atomic asegura que las operaciones bancarias sean ACID.
Factory	Creación de objetos Transaccion con estado inicial pendiente garantizando un invariante de dominio.
GRASP Controlador	Cada caso de uso (ej. RealizarTransferencia) es un controlador que orquesta puertos, entidades y adaptadores.
Observer	Señal post_save de Django: al crear un User, se dispara la creación automática del perfil Cliente.

4. Apéndices

4.1 Modelo de Datos (DER Resumido)

```
User (django.contrib.auth)
└─ 1:1
    Cliente
    └─ id (PK, auto)
    └─ usuario (OneToOneField → User, on_delete=CASCADE)
    └─ dni (CharField, unique, RegexValidator: ^\d{7,8}$)
    └─ telefono (CharField, max_length=20, blank=True)
    └─ direccion (TextField, blank=True)
    └─ fecha_nacimiento (DateField, null=True, blank=True)
    └─ genero (CharField, choices: M/F/N, default='N')
    └─ profesion (CharField, max_length=100, blank=True)
    └─ ingreso_mensual (DecimalField, null=True, blank=True)
    └─ nivel_educativo (CharField, choices: 5 niveles, default='secundario')
    └─ score_creditticio_inicial (IntegerField, default=500)
    └─ estado (CharField, default='activo')
    └─ fecha_registro (DateTimeField, auto_now_add)
    └─ @property nombre (devuelve first_name o username)
    └─ 1:N
        Cuenta
        └─ id (PK, auto)
        └─ cliente (FK → Cliente, on_delete=PROTECT)
        └─ tipo_cuenta (CharField, choices: ahorro/corriente)
        └─ saldo (DecimalField, max_digits=12, default=0.00)
        └─ moneda (CharField, default='USD')
        └─ alias (CharField, unique, nullable)
        └─ cvu (CharField, unique, nullable, 22 dígitos)
        └─ limite_transferencia_diario (DecimalField, default=1000.00)
        └─ fecha_apertura (DateTimeField, auto_now_add)
        └─ estado (CharField, choices: activa/inactiva/bloqueada)
        └─ 1:N (origen)
            Transaccion
            └─ id (PK, auto)
            └─ tipo (CharField, db_index - transferencia/deposito/retiro/...)
            └─ monto (DecimalField)
            └─ cuenta_origen (FK → Cuenta, null=True, PROTECT)
            └─ cuenta_destino (FK → Cuenta, null=True, PROTECT)
            └─ fecha_creacion (DateTimeField, auto_now_add, db_index)
            └─ estado (CharField, choices, db_index, default='completada')
            └─ descripcion (TextField, blank=True)
            └─ riesgo_fraude (FloatField, default=0.0)
            └─ es_fraude_confirmado (BooleanField, default=False)
            └─ 1:1
                AlertaFraude
                └─ id (PK, auto)
                └─ transaccion (OneToOneField → Transaccion, CASCADE)
                └─ score_riesgo (FloatField)
                └─ fecha_alerta (DateTimeField, auto_now_add)
                └─ accion_tomada (CharField, choices)
                └─ resuelta (BooleanField, default=False)
            └─ 1:N
                Tarjeta
                └─ numero (CharField, unique, RegexValidator: ^\d{16}$)
                └─ fecha_expiracion (DateField)
                └─ limite_credito (DecimalField, default=0.00)
                └─ saldo_actual (DecimalField, default=0.00)
                └─ estado (CharField, choices: activa/bloqueada/vencida)
                └─ CVV NO almacenado (PCI DSS)
```



4.2 Stack Tecnológico

Componente	Tecnología
Backend	Python 3.10+, Django 6.0
Base de datos	PostgreSQL 15+ (Neon Serverless)
Frontend	HTML5 semántico + CSS3 vanilla (sin frameworks)
Autenticación	Django Auth (PBKDF2, sesiones firmadas)
Machine Learning	Scikit-learn (Random Forest / Gradient Boosting)
Pruebas de carga	Locust
Control de versiones	Git + GitHub
Entorno	Virtualenv / venv
ORM	Django ORM con select_for_update() para concurrencia

4.3 Estado Actual y Próximos Pasos

Funcionalidades Implementadas

- ✓ Registro de cliente con todos los campos (DNI, género, nivel educativo, ingreso mensual, etc.)
- ✓ Inicio de sesión con protección CSRF
- ✓ Landing page pública con diseño responsivo (paleta azul marino)
- ✓ Dashboard con saldo, alias, CVU, movimientos recientes y resumen de productos
- ✓ Formulario de transferencia con pantalla de confirmación y lógica real
- ✓ Historial de movimientos con datos reales de la base de datos
- ✓ Solicitud de préstamos con simulación de cuotas (francés / alemán)

- ✓ Gestión de cuotas de préstamo con pago manual desde cuenta
- ✓ Alias y CVU en cuentas: búsqueda en transferencias y visualización en dashboard
- ✓ Modelo Tarjeta sin almacenamiento de CVV (PCI DSS)
- ✓ Modelos Prestamo, CuotaPrestamo, AlertaFraude, ConfiguracionSeguridad
- ✓ Señal post_save para creación automática de Cliente, Cuenta y ConfiguracionSeguridad
- ✓ Arquitectura híbrida MTV + Hexagonal con domain/, application/ y adapters/ implementados
- ✓ Accesibilidad WCAG 2.1 AA (contraste, ARIA, teclado, HTML semántico, skip link)
- ✓ Seguridad OWASP (CSRF, ORM, hashing, .env, rate limiting, logout con POST)

Funcionalidades en Desarrollo

- RF-07: Motor de detección de fraude con ML (Random Forest)
- RF-08: Autenticación de dos factores (2FA) con OTP
- RF-13: Constitución de plazo fijo
- RF-14: Cancelación anticipada de plazo fijo
- RF-15: Scoring crediticio con ML
- RNF-01: Pruebas de concurrencia con Locust
- RNF-04: Optimización de rendimiento y caché
- RNF-05: Validación completa de principios SOLID

Fin del documento — IEEE 830 v2.0 — 17/06/2026